Student Name	**Sarvesh Naik**
SRN No	31241943
Roll No	33
Program	Computer Science Engg.
Year	Third Year
Division	F
Subject	Artificial Intelligence Lab
Assignment No	5

Assignment 5

Problem statement: Implement the A* algorithm to solve the following problems given by HackerRank: a. N puzzle b. Robot Navigation problem c. Cities Distance (shortest path) problem

Objective:

To understand, implement, and apply the A* (A-star) search algorithm to solve different types of real-world and classical problems efficiently by finding the optimal (shortest or least-cost) path.

Theory:

2.1 N-Puzzle Problem (A Algorithm)

The N-Puzzle problem is a classic Artificial Intelligence problem. It consists of:

A $k \times k$ grid
$(k^2 - 1)$ numbered tiles
1 blank space (represented as 0)

The objective is to rearrange the tiles to reach the goal configuration by sliding tiles into the blank space.

Goal State (Example for 8-puzzle):

1 2 3
4 5 6
7 8 0

The blank space allows movement in four directions:

Up
Down
Left
Right

*2.2 A Search Algorithm**

A* is an informed search algorithm that finds the shortest path by combining:

Actual cost from start (g(n))
Heuristic estimate to goal (h(n))

It uses the evaluation function:
$f(n) = g(n) + h(n)$

It guarantees an optimal solution if the heuristic is admissible.

2.3 Heuristic Function

In this implementation, we use:

Misplaced Tiles Heuristic

Counts the number of tiles not in their correct position
h = 0 → Goal state reached
Lower h value → Closer to goal

Algorithm

Step 1: Start with the initial state
Step 2: Compute f(n) = g(n) + h(n)
Step 3: Insert the state into a priority queue
Step 4: Select the state with minimum f(n)
Step 5: Generate all possible neighboring states
Step 6: Compute heuristic for each neighbor
Step 7: Insert neighbors into the priority queue
Step 8: Repeat until goal state is reached
Step 9: Backtrack using parent nodes to obtain the final path

Python Implementation

```
from collections import deque

goal = "012345678"

# Moves: UP, DOWN, LEFT, RIGHT moves =
["UP", "DOWN", "LEFT", "RIGHT"]
dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]

def solve_puzzle(start):
    queue = deque([start])
    parent = {start: None}
    move_taken = {}

    while queue: curr =
        queue.popleft()

        if curr == goal:
```

```
            break zero_pos =

        curr.index('0') x, y =

        divmod(zero_pos, 3)


        for i in range(4): nx, ny = x +
            dx[i], y + dy[i]

            if 0 <= nx < 3 and 0 <= ny < 3:
                new_pos = nx * 3 + ny
                next_state = list(curr)

                # swap
                next_state[zero_pos], next_state[new_pos] = next_state[new_pos],
                next_state[zero_pos] next_state = ''.join(next_state)

                if next_state not in parent:
                    parent[next_state] = curr
                    move_taken[next_state] = moves[i]
                    queue.append(next_state)

    # Backtrack
    path path = []
    curr = goal

    while curr != start:
        path.append(move_taken[curr])
        curr = parent[curr]

    path.reverse()
    return path


# -------- Input
-------k = int(input())
start = ""

for _ in range(k * k):
    start += input().strip()

# -------- Solve -------result
= solve_puzzle(start)
```

```
# -------- Output
-------print(len(result))
for move in result:
    print(move)
```

Input :

Player: 1

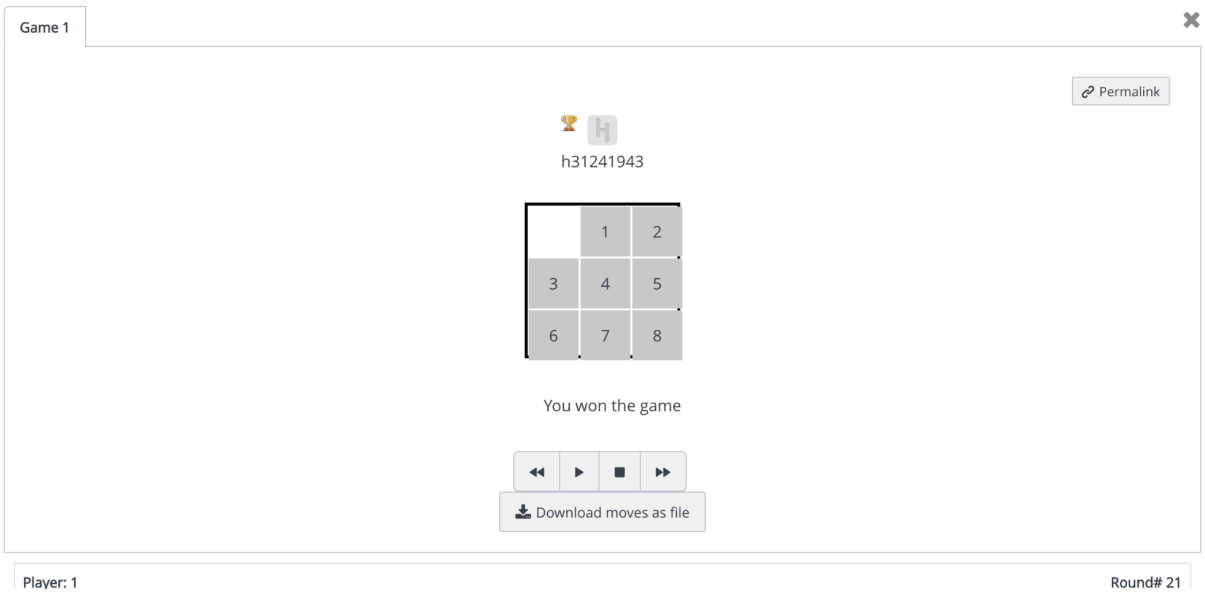
Input

```
3
0
3
8
4
1
7
2
6
5
```

Output:

Output

```
20
DOWN
DOWN
RIGHT
UP
LEFT
UP
RIGHT
DOWN
DOWN
RIGHT
UP
UP
LEFT
DOWN
DOWN
RIGHT
UP
LEFT
LEFT
UP
```



*2.4 Robot Navigation Problem (A Algorithm)**

The Robot Navigation problem involves finding the shortest path for a robot from a start position to a goal position in a grid environment.

The grid consists of:
- Free cells (0)
- Obstacles (1)

The robot can move in four directions:
- Up
- Down
- Left
- Right

---

*2.5 A Search Algorithm**

A* is used to find the shortest path efficiently by considering:

g(n): Cost from start node to current node h(n): Estimated cost from current node to goal f(n) = g(n) + h(n)

2.6 Heuristic Function

In this implementation, we use:

Manhattan Distance Heuristic h(n) = $|x_1 - x_2| + |y_1 - y_2|$ Suitable for grid-based movement
Ensures optimal path

Algorithm

Step 1: Initialize start node
Step 2: Compute f(n) = g(n) + h(n)
Step 3: Add node to priority queue
Step 4: Select node with lowest f(n)
Step 5: Explore neighboring cells
Step 6: Ignore blocked or visited nodes
Step 7: Update cost and parent
Step 8: Repeat until goal is reached
Step 9: Reconstruct path

Python Implementation

```
import heapq import

matplotlib.pyplot as plt import

numpy as np

# Heuristic (Euclidean for nice decimal values like your image)

def heuristic(a, b):

return round(((a[0]-b[0])**2 + (a[1]-b[1])**2)**0.5, 2)

def a_star(grid, start, goal):


open_list = []

heapq.heappush(open_list, (0, start))
```

```
came_from = {}

g = {start: 0}

while open_list:

_, current = heapq.heappop(open_list)

if current == goal:

path = [] while current in

came_from:

path.append(current) current

= came_from[current]

path.append(start) return

path[::-1] x, y = current

for dx, dy in [(0,1),(1,0),(0,-1),(-1,0)]:

nx_, ny_ = x+dx, y+dy if 0 <= nx_ < len(grid) and 0 <= ny_ < len(grid[0])

and grid[nx_][ny_] == 0:

temp_g = g[current] + 1 if (nx_, ny_) not in g or

temp_g < g[(nx_, ny_)]:

g[(nx_, ny_)] = temp_g f = temp_g +

heuristic((nx_, ny_), goal)

heapq.heappush(open_list, (f, (nx_,

ny_))) came_from[(nx_, ny_)] = current


return []

# Grid

grid = [
```

[0,0,0,0,0],

[1,1,0,1,0],

[0,0,0,1,0],

[0,1,0,0,0],

[0,0,0,1,0]

]

start = (0,0) goal = (4,4) path =

a_star(grid, start, goal) # --------

Visualization -------n = len(grid)

fig, ax = plt.subplots(figsize=(6,6))

for i in range(n): for j in range(n):

Colors

if (i,j) == start:

color = 'orange'

elif (i,j) == goal:

color = 'green' elif

grid[i][j] == 1:

color = 'black' elif

(i,j) in path: color

= 'yellow'

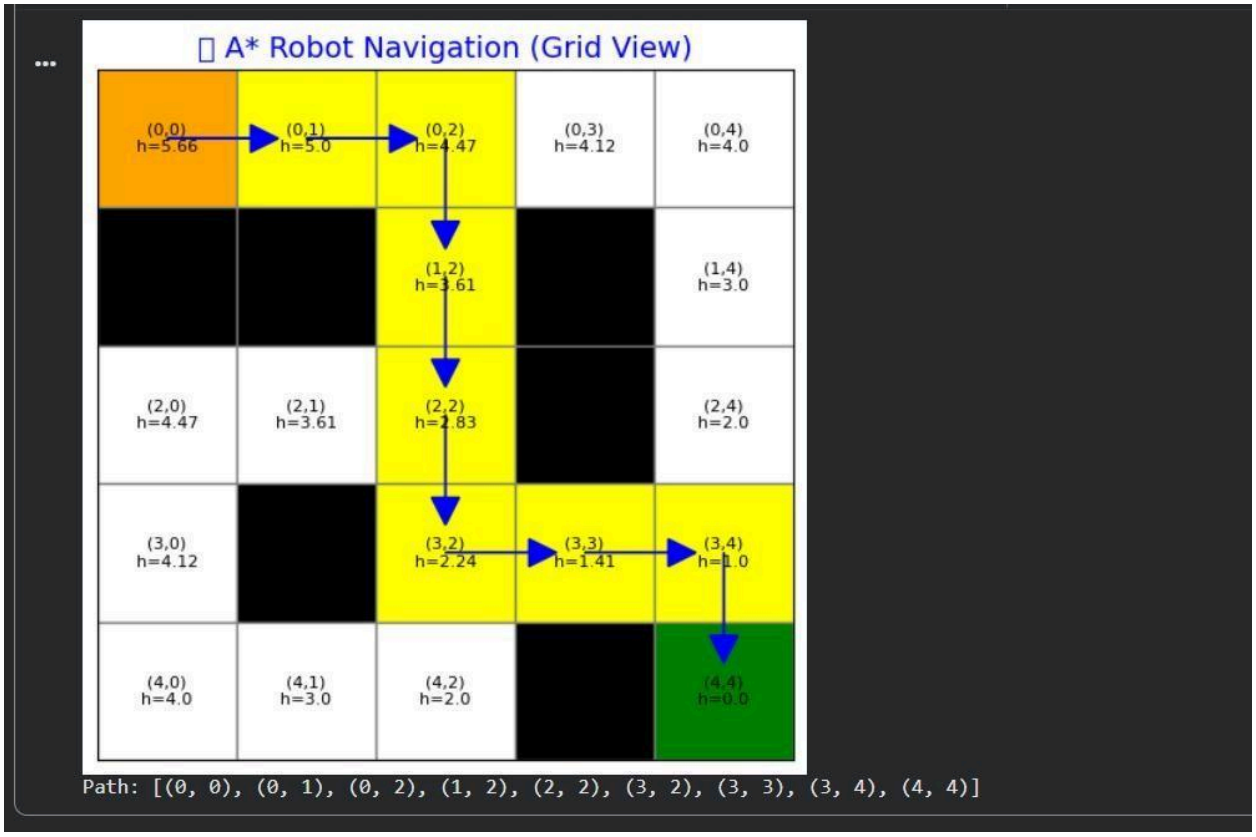
else:

color = 'white' rect = plt.Rectangle((j, n-i-1), 1, 1, facecolor=color,

edgecolor='gray') ax.add_patch(rect) # Heuristic text h =

```
heuristic((i,j), goal) ax.text(j+0.5, n-i-1+0.5, f"({i},{j})\nh={h}",
ha='center', va='center', fontsize=8)
# Draw arrows
for i in range(len(path)-1):
x1, y1 = path[i] x2, y2 = path[i+1]
ax.arrow(y1+0.5, n-x1-1+0.5,
(y2-y1)*0.6, -(x2-x1)*0.6,
head_width=0.2, head_length=0.2,
fc='blue', ec='blue') # Styling
ax.set_xlim(0, n) ax.set_ylim(0, n)
ax.set_xticks([])

ax.set_yticks([]) plt.title("       A* Robot Navigation (Grid View)",
fontsize=14, color='blue') plt.show() print("Path:", path)
```

Output :



*2.7 Cities Distance Problem (Shortest Path using A)**

The Cities Distance problem involves finding the shortest path between cities represented as a graph.

The graph consists of:

Nodes → Cities

Edges → Distance between cities

---

*2.8 A Search Algorithm**

A* is used to determine the shortest path by combining:

Actual travel cost (g(n))

Estimated cost to destination (h(n))

f(n) = g(n) + h(n)

2.9 Heuristic Function

In this implementation, we use:

Estimated Distance Heuristic Predefined heuristic values for each city h(n) = Estimated distance to goal city h(goal) = 0

Algorithm

Step 1: Start from initial city
Step 2: Compute f(n) = g(n) + h(n)
Step 3: Insert into priority queue
Step 4: Select city with lowest f(n)
Step 5: Explore neighboring cities
Step 6: Update cost and parent
Step 7: Repeat until destination is reached
Step 8: Backtrack to get shortest path

Python Implimentation :

```
import heapq import

matplotlib.pyplot as plt

import networkx as nx

# ------------------------------

#      NEW GRAPH (Different Values)

# ------------------------------

graph = {

'S': {'A': 3, 'B': 2},

'A': {'S': 3, 'C': 4, 'D': 6},

'B': {'S': 2, 'D': 3},

'C': {'A': 4, 'G': 5},
```

```
'D': {'A': 6, 'B': 3, 'G': 4},
'G': {'C': 5, 'D': 4}
}
# --------------------------------
#     NEW HEURISTIC VALUES
# --------------------------------
heuristic = {
'S': 7,
'A': 6,
'B': 5,
'C': 3,
'D': 2,
'G': 0
}
# --------------------------------
#     HEURISTIC TABLE
# --------------------------------def
show_heuristic_table(): fig, ax
= plt.subplots() ax.axis('off')

data = [[node, heuristic[node]] for node in heuristic]
table = ax.table( cellText=data, colLabels=["City",
"Heuristic h(n)"], loc='center', cellLoc='center'
```

) table.auto_set_font_size(False)

table.set_fontsize(12) table.scale(1.5, 2) for

(row, col), cell in table.get_celld().items():

if row == 0:

cell.set_facecolor('#FF7043') # orange header

cell.set_text_props(color='white', weight='bold')

else:

cell.set_facecolor('#FFF3E0') # light orange plt.title("✨

Heuristic Table", fontsize=16, color="#E64A19") plt.show()

⭐ A* Algorithm

------------------------------def

a_star(start, goal):

pq = []

heapq.heappush(pq, (heuristic[start],

start)) g = {start: 0} parent = {} visited =

set() while pq:

f, node =

heapq.heappop(pq) if node

in visited: continue

visited.add(node) if node ==

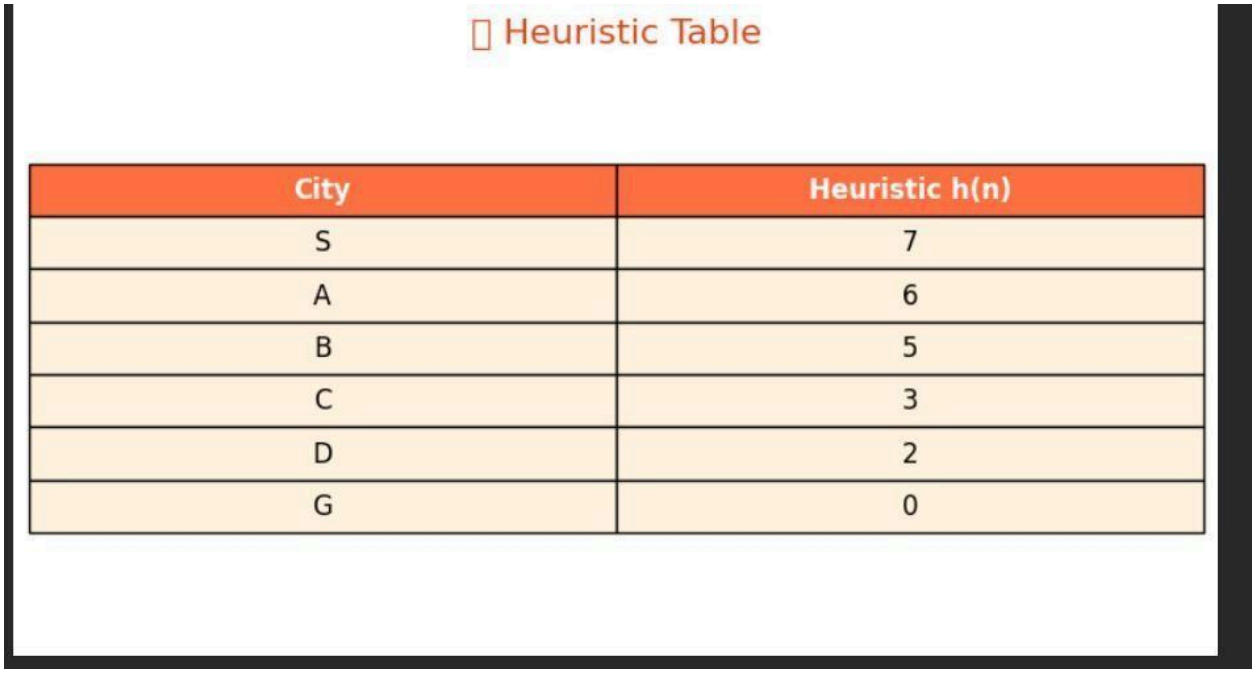
goal:

```
break for neighbor, cost in
graph[node].items(): temp_g = g[node] +
cost if neighbor not in g or temp_g <
g[neighbor]: g[neighbor] = temp_g f_val =
temp_g + heuristic[neighbor]
heapq.heappush(pq, (f_val, neighbor))
parent[neighbor] = node # Path
reconstruction path = [] node = goal while
node in parent: path.append(node)
node = parent[node] path.append(start)
path.reverse() print("\n✨ A* Path:", "
→ ".join(path)) print("           Total
Cost:", g[goal]) draw_graph(path)
# ------------------------------
#     BEAUTIFUL GRAPH
# ------------------------------
def
draw_graph(path): G
= nx.Graph() for u in
graph: for v in
graph[u]:
G.add_edge(u, v, weight=graph[u][v])
pos = nx.spring_layout(G, seed=10)
```

```
plt.figure(figsize=(10,7))
# Base graph nx.draw(G, pos,
with_labels=True,
node_color='#FFE082', # soft
yellow node_size=2200,
font_size=12, font_weight='bold')

# Edge labels nx.draw_networkx_edge_labels(
G, pos, edge_labels=nx.get_edge_attributes(G,
'weight'), font_color='blue'
)
# Path highlight path_edges =
list(zip(path, path[1:]))
nx.draw_networkx_edges( G, pos,
edgelist=path_edges,
edge_color='#D32F2F', width=4
)
nx.draw_networkx_nodes( G,
pos, nodelist=path,
node_color='#81C784', #
green node_size=2400
)
```

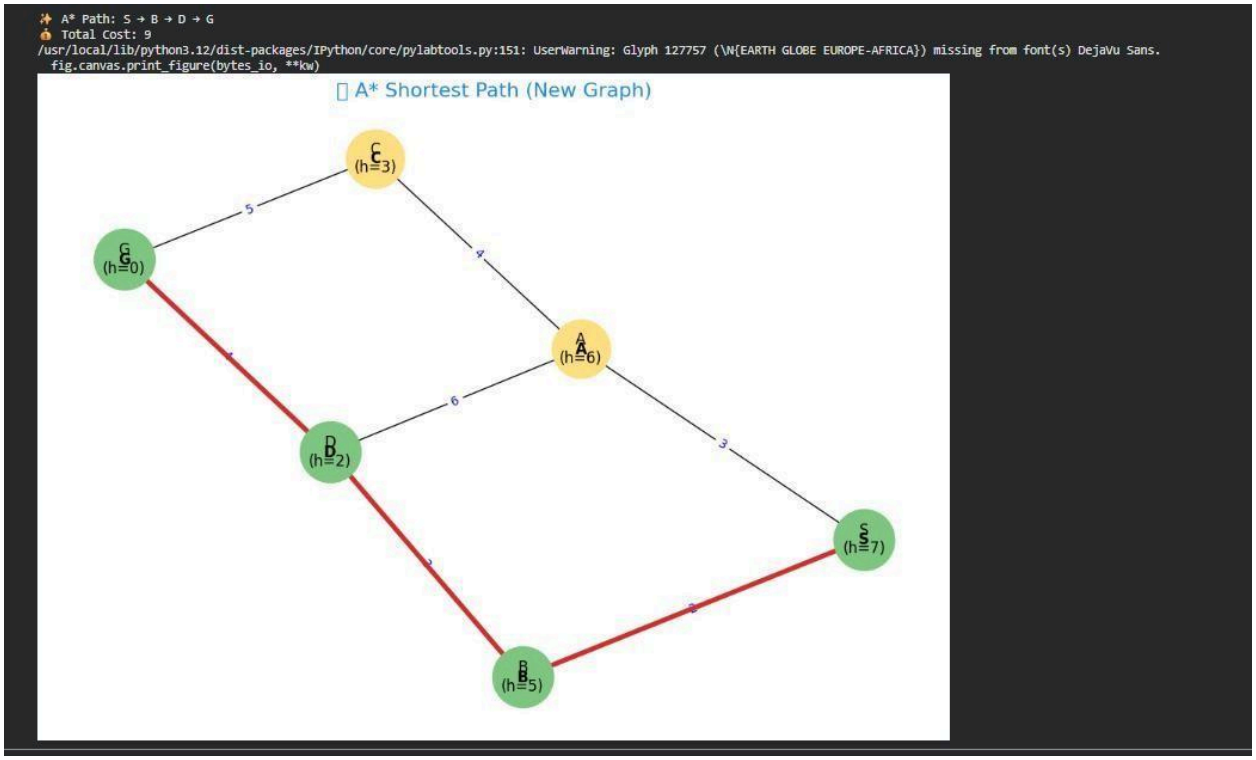
```
# Labels with heuristic labels = {node:
f"{node}\n(h={heuristic[node]})" for node in G.nodes()}
nx.draw_networkx_labels(G, pos, labels)

plt.title("          A* Shortest Path (New Graph)", fontsize=16, color="#1E88E5")
plt.show()
# ------------------------------
# RUN
#
------------------------------show_
heuristic_table() a_star('S', 'G')
```

Output :



 Heuristic Table

City	Heuristic h(n)
S	7
A	6
B	5
C	3
D	2
G	0



Conclusion

In this experiment, the A* search algorithm was successfully implemented to solve different types of problems, namely the N-Puzzle problem, Robot Navigation problem, and Cities Distance (shortest path) problem. The algorithm efficiently combines both actual cost (g(n)) and heuristic cost (h(n)) to determine the optimal path.

For the N-Puzzle problem, A* helped in finding the sequence of moves required to reach the goal state using heuristic guidance. In the Robot Navigation problem, it efficiently determined the shortest path in a grid while avoiding obstacles. In the Cities Distance problem, it was used to compute the shortest path between cities in a weighted graph.

The use of appropriate heuristic functions such as misplaced tiles and distance-based heuristics improved the performance of the algorithm. The visualization of graphs and grids further helped in understanding the traversal and decision-making process of A*.

Overall, A* proved to be an optimal, complete, and efficient search algorithm, making it highly suitable for real-world applications such as pathfinding, robotics, and navigation systems.